Ders 10 Genişletilmiş Notu: Tek Çevrimli Bilgisayar, Komut Çözücü, Pipeline ve Tehlikeleri, G/Ç Arayüzü ve Haberleşme

Ders 10 Genişletilmiş Notu: Tek Çevrimli Bilgisayar, Komut Çözücü, Pipeline ve Tehlikeleri, G/Ç Arayüzü ve Haberleşme

> Bu not, ders kaydının altyazısından hazırlanmış ayrıntılı ders notudur. Kısa tekrar için aynı dersin kısa notuna bakın.

İçindekiler

- 1. Dersin Planı
- 2. Datapath Tekrarı
- 3. Tek Çevrimli Bilgisayar ve Komut Kümesi
- 4. Bellek İçeriğinden Komut Çözme
- 5. Kontrol Birimi ve Komut Çözücü
- 6. Tek Çevrimli Bilgisayarın Çevrim Süresi
- 7. Pipeline Datapath: Üç Aşamalı Yapı
- 8. Kontrol Birimiyle Dört Aşamalı Pipeline
- 9. Pipeline Tehlikeleri
- 10. Giriş/Çıkış Birimleri ve Çevre Birimleri
- 11. G/Ç Arayüzü ve Portlara Erişim
- 12. Örnek Arayüzün İç Yapısı ve Adres Çözme
- 13. Arayüz ile Cihaz Arasında Strobing ve Handshaking
- 14. Paralel ve Seri İletişim

1. Dersin Planı

Hoca, önceki derste merkezi işlem birimi içindeki aritmetik lojik birimi (ALU) tasarladıklarını, artık farklı aritmetik ve lojik işlemleri yapan blokların tasarlanabildiğini hatırlatarak başladı. Bu derste işlemci tamamlanacak, yani kontrol bloğu tasarlanacak ve ardından işlemciyi hızlandıran teknolojilere geçilecektir. Bunun ilk örneği, işlemci içindeki işleri paralelleştirip arka planda hızlandıran boru hattı işlemedir (pipeline processing).

Haftanın asıl konusu ise giriş/çıkış ve iletişimdir (input/output and communications). Hoparlör, mikrofon, klavye gibi çevre birimlerinin işlemciyle nasıl birlikte çalıştığı ve aralarındaki iletişimin nasıl kurulduğu konuşulacak. Bu başlıkta doğrudan bellek erişimi (direct memory access) ve kesme (interrupt) ile haberleşme de gündeme gelecektir. Sonraki haftalarda önbellek (cache memory) organizasyonları ile sanal bellek (virtual memory) gelecek. Hoca sanal bellek için şu soruyu öne çıkardı: Hard disk üzerindeki veriyi işlemci nasıl görüyor ve nasıl erişiyor? Bu kavram işletim sistemleri dersinde de görülecek, burada ise tasarım aşamasındaki işleyişi anlatılacaktır.

Hoca dersi ikiye böldü: İlk yarıda CPU tasarımı ve pipeline, ikinci yarıda giriş/çıkış ve iletişim.

2. Datapath Tekrarı

Yeni konuya geçmeden önce hoca geçen derste tasarlanan datapath birimini (datapath unit) kısaca hatırlattı.

2.1 Genel Yapı

Datapath'in iki büyük bloğu vardır: yazmaç dosyası (register file) ve fonksiyon birimi (function unit). İç yazmaçlardan (internal register) kaynak işlenenler (source operand) çoklayıcılarla (multiplexer) seçilir. Seçilen kaynaklar arasında, aritmetik lojik birim ile kaydırma birimi (shifter unit) üzerinde istenen işlem yapılır. Sonuç D-Multiplexer üzerinden bütün yazmaçların girişine bağlanır. Sonucun hangi yazmaca yazılacağına hedef seçimi (destination select) karar verir; sonuç yazmaca kaydedilebilir de kaydedilmeyebilir de.

> **Hocanın vurgusu:** Bu bloğun nasıl çalıştığını artık "gözünüz kapalı" biliyor olmalısınız. Hocaya göre bu kısmı bilmek, dersin yaklaşık yüzde otuzunu bilmek demektir.

2.2 Fonksiyon Birimi

N bitlik aritmetik lojik birim, A ve B sayıları arasında seçilen işlemi yapar ve sonucu G çıkışında üretir. Aritmetik kısım paralel toplayıcıyla (parallel adder) kurulur. Paralel toplayıcı tam toplayıcılardan (full adder) oluşur: Dört tam toplayıcı dört bitlik bir paralel toplayıcı verir. Y girişinin önüne yerleştirilen B giriş mantığı (B input logic) sayesinde, A ve B sayıları farklı işlemlerden geçirilerek farklı aritmetik fonksiyonlar elde edilir.

Lojik işlemler çoklayıcılı bir blokla yapılır. Slayt bunu tek bit için gösterir. N bitlik tasarım, n adet dörtten bire çoklayıcı ve kapıların n kopyasını gerektirir.

Aritmetik ve lojik blokları birleştiren, yüksek anlamlı mod seçme ucu S_2'dir (mode select). S_2, çıkış katmanındaki ikiden bire çoklayıcının seçme ucudur. Aslında iki blok da paralel olarak işlem yapar, fakat yalnızca birinin sonucu çıkışa aktarılır.

2.3 Kaydırma Birimi

Kaydırma birimi de çoklayıcılarla kuruldu. Derste dört bitlik bir veri yolu üzerinden örneklendi. Dörtten bire çoklayıcı kullanılır, ama yalnızca üç girişi aktiftir:

Seçme uçları	İşlem
00	Doğrudan transfer
01	Bir bit sağa kaydırma
10	Bir bit sola kaydırma

Benzer bir çoklayıcı yapısıyla bir, iki veya üç konum döndürme (rotate) yapabilen birim de tasarlanabilir.

> **Hocanın vurgusu:** Aritmetik işlemleri yapan paralel toplayıcıda, lojik işlemleri yapan çoklayıcılı yapıda ve kaydırma biriminde hiçbir senkron devre elemanı yoktur. Hepsi kombinasyonel devredir. Çünkü amaç, bir saat periyodunda bir mikro işlem yapan tek çevrimli bilgisayarı (single cycle computer) tasarlamaktır.

2.4 Blok Şeması ve Boyutlar

Yazmaç dosyasının boyutu $2^m \times n$'dir: 2^m tane yazmaç vardır ve her biri n bittir. Bu yüzden veri yollarının genişliği n bit olarak işaretlenir. Örneğin $2^3 = 8$ yazmaç varsa, buradaki $m = 3$ kaynak seçme ve hedef seçme alanlarının kaç bit olacağını da söyler.

Blokta adres çıkışı (address out) ve veri çıkışı (data out) belleğe erişim için kullanılır. Bunlar giriş/çıkış için de benzer şekilde kullanılabilir. Bellekten okunan veri ile fonksiyon birimi sonucu arasındaki seçimi D-Multiplexer yapar. Fonksiyon birimi ayrıca işlem sonucuna göre durum bitlerini üretir: taşma (overflow), elde (carry), negatif (negative) ve sıfır (zero).

2.5 Kontrol Kelimesi

Sekiz adet n bitlik yazmaçlı yapıda kontrol uçları belli bir sırada dizilerek 16 bitlik bir kontrol kelimesi (control word) oluşturulur:

Alan	Görev	Bit
DA	Hedef yazmaç adresi	3
AA	A kaynak yazmacı adresi	3
BA	B kaynak yazmacı adresi	3
MB	B-Multiplexer seçimi	1
FS	Fonksiyon seçimi	4
MD	D-Multiplexer seçimi	1
RW	Yazmaca yazma	1

Toplam: $3 + 3 + 3 + 1 + 4 + 1 + 1 = 16$ bit.

Her alanın kodlaması tablolarda verilmiştir. R0 için 000, R7 için 111 yazılır. MB = 0 ise B girişi yazmaçtan, MB = 1 ise sabit sayı girişinden alınır. MD, bellek ile fonksiyon birimi arasında seçim yapar. RW gereklidir, çünkü bütün işlem sonuçları yazmaca yazılmaz: Belleğe yazma yapılıyorsa sonuç bir yazmaca yazılmaz.

Hoca, tabloların ezberlenmesinin gerekmediğini söyledi. Önemli olan, verilen bir mikro işlem için 16 kontrol ucuna ne uygulanacağını bulabilmektir. Geçen dersteki örneklerde bu önce sembolik gösterimle (using symbolic notation), sonra ikili kontrol kelimeleriyle (using binary control words) yazılmıştı.

> **Hocanın vurgusu:** Kontrol kelimesi soruları, hocaya göre klasik sınavlarda sık sorulan ve en kolay sorulardandır.

3. Tek Çevrimli Bilgisayar ve Komut Kümesi

Elimizde artık bir datapath var. Bu haftanın konusu, onun için gerekli kontrol devresini tasarlamak ve tek çevrimli bilgisayarı tamamlamaktır.

3.1 Bileşenler

Yapıda iki bellek alanı vardır: komut belleği (instruction memory) ve veri belleği (data memory). Program sayacı (program counter, PC), o anda yürütülen komutun bellekteki adresini gösterir. Yazmaç dosyası ise datapath içinde tasarlanan bloktur. Datapath tek başına yalnızca mikro işlemler yapabilir. Bu mikro işlemler bir araya getirilerek komutlar (instruction) oluşturulur.

3.2 Komut Formatı

Her komut 16 bittir ve bitler 0'dan 15'e numaralandırılır. Bütün adresleme modlarında ortak olan, yüksek anlamlı 7 bitin (15-9) işlem kodu (opcode) olmasıdır. Kalan 9 bit, 3'er bitlik üç alana ayrılır ve kullanımı adresleme moduna göre değişir: $7 + 3 + 3 + 3 = 16$.

Hangi adresleme modunda çalışıldığını opcode söyler. Opcode yalnızca işlemin toplama mı, artırma mı, taşıma mı olduğunu değil, işlemin hangi modda yapılacağını da gösterir. Mod anlaşıldıktan sonra kalan alanların nasıl yorumlanacağı bilinir.

Soru: Kaynak yazmaç A alanı üç formatta da aynı değil mi? **Cevap:** Evet, üç formatta da aynı adla ve aynı uzunlukta (3 bit) yer alır. Yalnız kullanım amacı bazılarında farklıdır.

3.3 Adresleme Modları

Basit bir model kurulduğu için üç adresleme modu vardır:

- **Yazmaç adresleme (register addressing):** İki yazmaç arasında işlem yapılır. İki kaynak alanı (SA, SB) işleme katılan yazmaçları, hedef alanı (DR) sonucun yazılacağı yazmacı seçer. Örnek: $R5 \leftarrow R1 + R3$.
- **İvedi adresleme (immediate addressing):** Komutun içinde bir işlenen vardır. Örnek: $R3 \leftarrow R2 + 3$. Buradaki 3 komutun içinde yer alır.
- **Atlama ve dallanma adresleme (jump and branch addressing):** Koşullu ya da koşulsuz olarak bir adrese dallanılır. Koşul olarak SA yazmacı kullanılabilir (örneğin içeriği

sıfırsa). Koşul sağlanırsa PC'ye, PC artı adres şeklindeki göreli (relative) adres yüklenir. Yani yürütülen komutun belli miktar yukarısındaki ya da aşağısındaki satıra gidilir.

İvedi işlenen için yalnızca 3 bit ayrılmıştır, bu yüzden 0 ile 7 arasında değer alabilir. Sebep, sabit uzunluklu komut formatıdır (fixed length instruction). Hocanın karşılaştırması: Intel 8086'da komut uzunlukları değişkendir (variable length instruction). Burada ise sade bir yapı kurmak için sabit uzunluk seçilmiş ve bu, ivedi işlenenin alabileceği değerlere kısıt koymuştur.

Dallanma adresi, adres sol (address left) ve adres sağ (address right) denen 3 bitlik iki alanın yan yana getirilmesiyle (concatenate) oluşan 6 bitlik sayıdır.

> **Ders sonrası not:** Hoca sign extend anlatımında bu iki alana "source A adresi ve source B adresi" dedi; doğrusu bu değildir. SA alanı dallanmada test edilen yazmacı tutar, adres alanı ise onun dışındaki iki alandan oluşur. Hocanın kendi BRZ örneği bunu gösterir: SA = 110 (R6) iken adres 101 ile 100'den oluşur. Adres SA ve SB'den oluşsaydı 110100, yani -12 çıkardı; derste bulunan -20 ise 101100'e karşılık gelir. Ayrıca hocanın burada kullandığı "toplanıyor" sözü aritmetik toplama değil, yan yana yazmadır.

3.4 Komut Tablosu

Komut tablosunda (instruction specifications) yalnızca 19 komut vardır. Hoca bunu "8086 komut tablosu gibi değil" diye yorumladı. Kısa notta slayttan aktarılan satırlar şunlardır:

Komut	Opcode	Sembol	İşlem
Move A	0000000	MOVA	$R[DR] \leftarrow R[SA]$
Increment	0000001	INC	$R[DR] \leftarrow R[SA] + 1$
Add	0000010	ADD	$R[DR] \leftarrow R[SA] + R[SB]$
Subtract	0000101	SUB	$R[DR] \leftarrow R[SA] - R[SB]$
Load Immediate	1001100	LDI	$R[DR] \leftarrow \text{zf}\, OP$
Add Immediate	1000010	ADI	$R[DR] \leftarrow R[SA] + \text{zf}\, OP$
Load	0010000	LD	$R[DR] \leftarrow M[R[SA]]$
Store	0100000	ST	$M[R[SA]] \leftarrow R[SB]$
Branch on Zero	1100000	BRZ	$R[SA] = 0$ ise $PC \leftarrow PC + \text{se}\, AD$
Branch on Negative	1100001	BRN	$R[SA] < 0$ ise $PC \leftarrow PC + \text{se}\, AD$
Jump	1110000	JMP	$PC \leftarrow R[SA]$

Bunların yanında azaltma (DEC), lojik işlemler (AND, OR, XOR, NOT), B yazmacı üzerinde taşıma (MOVB) ve sağa ve sola kaydırma (SHR, SHL) komutları vardır. Artırma komutunda kaynak yazmaç, hedef yazmaçtan farklı olabilir. ADD ise iki yazmacı toplayıp sonucu üçüncü bir yazmaca yazabilir.

Tablodaki yıldız işareti için slayttaki açıklama şudur: Bu komutlarda ayrıca $PC \leftarrow PC + 1$ yapılır. Yani PC, komut her yürütüldüğünde bir artırılarak bir sonraki komuta geçilir.

> **Ders sonrası not:** Dallanma gerçekleştiğinde ve JMP'de PC'ye yeni adres yüklendiği için, ayrıca +1 uygulanması büyük olasılıkla söz konusu değildir. +1 kuralı büyük olasılıkla dallanmayan akış içindir; altyazıda bu ayrım açıkça söylenmedi.

Bellek komutlarında dikkat edilecek nokta, hangi alanın neyi tuttuğudur:

> **Hocanın vurgusu:** LD ve ST'de bellek adresi SA yazmacında tutulur. ST'de belleğe yazılacak veri SB yazmacındadır. Buna dikkat edin.

3.5 Zero Fill ve Sign Extend

Zero fill: İvedi işlenen 3 bittir, ama 16 bitlik bir yazmaca yüklenecek ya da 16 bitlik bir yazmaçla işleme girecektir. Bu yüzden LDI ve ADI'de yüksek anlamlı bitlere 0 doldurularak 16 bite genişletilir.

Sign extend: Dallanmada $PC \leftarrow PC +$ se AD yapılır. PC 16 bittir, adres ise 6 bit. Toplanabilmeleri için aynı uzunlukta olmaları gerekir. Fakat adres işaretli (signed) bir sayıdır: Yüksek anlamlı bit 0 ise pozitiftir ve ileri gidilir, 1 ise negatiftir ve geri gidilir. Bu yüzden genişletme işaret bitiyle yapılır. Adres negatifse, 16 bite genişletirken üstteki 10 bitin tamamına 1 yazılır.

> **Hocanın vurgusu:** Zero fill ile sign extend birbirinden ayrı işlemlerdir. Sign extend, 6 bitlik adresin yüksek anlamlı bitine bakar ve onu üst bitlere doldurur.

4. Bellek İçeriğinden Komut Çözme

Slayttaki örnekte (memory representation of instructions and data) bir bellek içeriği verilir ve PC'nin gösterdiği adresteki komutlar çözülür. Her seferinde yüksek anlamlı 7 bite bakılır, kalan alanlar buna göre yorumlanır.

Adres 25. Opcode 0000101, yani 5'tir. (Hoca sınıfa sordu: "Opcode'u kaç?" Bir öğrencinin "5" demesiyle devam edildi.) Tablodan bunun SUB olduğu görülür. Yazmaç adresleme kullanıldığı için alanlar sırasıyla DR, SA, SB'dir: DR = 1, SA = 2, SB = 3. Komut: $R1 \leftarrow R2 - R3$.

Adres 35. Opcode 0100 ile başlar. Tabloda bunun ST olduğu görülür. ST, B yazmacındaki değeri A yazmacının gösterdiği bellek gözüne yazdığı için iki alan da önemlidir: SA = 4, SB = 5. Komut: $M[R4] \leftarrow R5$.

Adres 45. Opcode çözüldüğünde ADI çıkar. DR = 2, SA = 7, ivedi işlenen 3'tür. Komut: $R2 \leftarrow R7 + 3$.

Adres 55. Opcode BRZ'dir. SA alanı 6'yı gösterir, yani test edilen yazmaç R6'dır. Adres alanı 3 bitlik `101` ve `100` sayılarının birleşimidir:

- Birleştirme: `101` + `100` → `101100`.
- İşaret biti 1 olduğu için sayı negatiftir.
- Büyüklüğü bulmak için ikiye tümleyen alınır. Hocanın kestirme yolu: Sağdan başla, ilk 1'e kadar (o dahil) bitleri aynen yaz, sonrakilerin tersini al. Sağdaki `100` aynen kalır, soldaki `101` ise `010` olur. Sonuç `010100`.
- Değer: $2^4 + 2^2 = 16 + 4 = 20$. Demek ki AD = -20.

Sonuç: $R6 = 0$ ise $PC \leftarrow PC - 20$. Yani 20 adres geriye dallanılır. 16 bite sign extend edilmiş hali `1111 1111 1110 1100` olur (ara adım eklendi).

> **Hocanın vurgusu:** İşaret bitiyle ilgili bu açıklama önemlidir.

Adres 70. Bu adres bir komut değil, veri alanı olarak verilmiştir. Slayt, 35 adresindeki komut yürütüldükten sonra bu gözün değerini gösterir. Hoca bunu geçti.

5. Kontrol Birimi ve Komut Çözücü

Datapath daha önce tasarlanmıştı. Görülmeyen kısım kontrol birimidir (control unit). Amaç, şekildeki mavi blok olan kontrol devresini, yani komut çözücüyü (instruction decoder) tasarlamaktır.

5.1 Komutun Yolu ve Yeni Kontrol İşaretleri

PC komut belleğinin adres bilgisini oluşturur. PC'nin gösterdiği adresten 16 bitlik komut çekilir ve çözücüye gönderilir. Çözücünün görevi, gelen komuttan kontrol kelimesini üretmektir. Kelimede daha önce görülen MB, FS, MD, RW gibi alanlar ve adres alanları bulunur. Bunlara dört yeni işaret eklenir:

İşaret	Anlamı	Değerler
MW	Belleğe yazma (memory write)	Yazma var mı, yok mu
PL	PC'ye yükleme (program counter load)	Yükleme var mı, yok mu
JB	Atlama mı, dallanma mı (jump/branch)	0: koşullu dallanma, 1: atlama
BC	Dallanma koşulu (branch condition)	0: sıfırsa dallan, 1: negatifse dallan

BC'nin seçtiği koşul, A yazmacının içeriğine göre sınanır.

5.2 PC'ye Yükleme Yolları

Hoca PC'ye adres yüklemenin iki ayrı kanaldan olabildiğine dikkat çekti. Kısa nottaki slayt satırlarına göre bunlar dallanmada kullanılan $PC+$se AD ve JMP'de A yolundan gelen yazmaç değeridir ($PC \leftarrow R[SA]$).

5.3 Komut Çözücünün Doğruluk Tablosu

Çözücü, komut tablosundaki komutları gruplara ayırarak kurulur: fonksiyon birimi üzerinde işlem yapanlar, bellek okuma ve yazma yapanlar, dallanma komutları ve atlama komutları. Girişler olarak opcode'un 15, 14, 13 ve 9. bitleri kullanılır. Çıkışlar MB, MD, RW, MW, PL, JB ve BC'dir. Slayttaki doğruluk tablosuna (truth table for instruction decoder logic) göre bu çıkışlar birer Boole fonksiyonu olarak bulunur ve şekildeki AND, NOT gibi kapılar elde edilir. Bu yapıya sabit kablolu kontrol (hardwired control) denir.

En kolay örnek MB'dir. Şekilde MB ucu doğrudan opcode'un 15. bitinden gelir: $MB = I_{15}$. Tabloda da iki sütunun aynı olduğu görülür. Bu sonuç Karnaugh haritasına taşıyarak da bulunabilir.

5.4 RW Çıkışının Karnaugh Haritası

Derste RW çıkışı birlikte çözüldü. Giriş bitleri $I_{15}, I_{14}, I_{13}, I_9$'dur. Tablodan okunan değerler:

$I_{15}\, I_{14}\, I_{13}\, I_9$	RW
0 0 0 X	1
0 0 1 X	1
0 1 0 X	0
1 0 0 X	1
1 1 0 0	0
1 1 0 1	0
1 1 1 X	0

X olan satırlar haritada ikişer göze yazılır. Tabloda yer almayan gözlere fark etmez (don't care, d) konur:

$I_{15}I_{14} \setminus I_{13}I_9$	00	01	11	10
00	1	1	1	1
01	0	0	d	d
11	0	0	0	0
10	1	1	d	d

Hoca "kaçlı grup çıkar?" diye sordu. Bir öğrenci en üst satırla en alt satırın birleştirilebileceğini söyledi. Bu iki satır, fark etmez gözleri de 1 sayılarak sekizli grup oluşturur. Bu grupta $I_{14} = 0$ sabittir, diğer değişkenler değişir. Sonuç:

$$RW = \overline{I_{14}}$$

Devreye bakıldığında da RW'nin I_{14}'ün değili olduğu görülür.

> **Ders sonrası not:** Hoca tabloda yer almayan "iki tane göz" dedi. Doğrusu iki satırdır: 011X ve 101X, yani haritada dört göz fark etmezdir. 16 gözden 12'si belirlidir. Python ile bütün belirli gözler kontrol edildi: Tek harfli çözüm yalnızca $\overline{I_{14}}$'tür.

> **Hocanın vurgusu:** Doğruluk tablosunu inceleyin. Her çıkışın fonksiyonunu girişlere göre bulun ve şekildeki devreyle aynı elemanlardan oluştuğunu görün.

Hoca, sabit kablolu kontrolün RISC işlemcilerde kullanıldığını ve "software kontrol devresine" göre daha hızlı olduğu için tercih edildiğini söyledi.

> **Ders sonrası not:** Hocanın "software kontrol" dediği yapının adı büyük olasılıkla mikroprogramlı kontroldür (microprogrammed control). Altyazıda bu ad geçmedi.

6. Tek Çevrimli Bilgisayarın Çevrim Süresi

Tek çevrimli bilgisayarda bir saat periyodu içinde bir mikro işlem tamamlanır. Peki bir çevrimin süresi en az kaç olabilir? Bunun için PC'ye ilk adresin yüklendiği andan başlayarak arka arkaya yapılan bütün işlemlerin süresine bakılır. Kullanılan malzemenin ve teknolojinin hızına göre slayttaki gecikmeler şöyledir:

Adım	Gecikme (ns)
PC'ye değerin yüklenmesi	0.2
Komut belleğinden komutun çekilmesi	4
Komut çözme, kontrol kelimesi ve kaynak işlenenleri okuma	0.6
B-Multiplexer	0.2
Fonksiyon birimi	4
D-Multiplexer	0.2
Sonucun yazmaca yazılması	0.6

$$T_{\text{min}} = 0.2 + 4 + 0.6 + 0.2 + 4 + 0.2 + 0.6 = 9.8 \text{ ns}$$

Hoca toplarken "9 oldu, 9.8 mi oldu?" diye duraksadı; sonuç 9.8 ns'dir (python ile doğrulandı). Hoca, 4 ns'lik iki değerin diğerlerine göre çok büyük olduğuna dikkat çekti. Bu periyot en kısa saat periyodudur ve işlemcinin saat frekansını belirler.

7. Pipeline Datapath: Üç Aşamalı Yapı

CPU tasarlandı. Şimdi soru şudur: Bu işlemcinin çalışmasını nasıl hızlandırabiliriz?

> **Hocanın vurgusu:** CPU içinde bir kontrol devresi ve bir datapath birimi vardır. Datapath içinde yazmaçlar, ALU ve kaydırma birimi bulunur. Kontrol birimi datapath'e 16 bitlik kontrol kelimesini gönderir, datapath ise durum bitlerini geri yollar. Hocaya göre ders bittikten yıllar sonra akılda kalması gereken temel konu budur.

"Pipeline datapath" denince, bu yapı içindeki datapath biriminin işlemlerinin nasıl paralelleştirileceği kastedilir.

7.1 Geleneksel Yöntem

Geleneksel (conventional) tasarımda bloklar arka arkaya çalışır ve toplam süre tek bir saat periyoduna sığmak zorundadır. Bu örnekte süreler az önceki tasarımdan farklıdır:

$$T = 0.6 + 0.2 + 0.8 + 0.2 + 0.6 = 2.4 \text{ ns}$$

Bu toplamda 0.6 yazmaçtan okuma, 0.2 çoklayıcı, 0.8 fonksiyon birimi, 0.2 D-Multiplexer, 0.6 yazmaca yazmadır.

7.2 Aşamalara Bölmek

Her katmanın sonuna bir yazmaç eklenir. Yazmaçlar katmanları birbirinden ayırır ve bu sayede aşamalar (stage) paralel çalışabilir. Bu yapıya pipeline denir. Üç aşama vardır:

Aşama	Yaptığı iş	Süre (ns)
OF (operand fetch)	Yazmaçtan okuma ve B-Multiplexer	$0.6 + 0.2 = 0.8$
EX (execution)	Fonksiyon birimi	1
WB (write back)	D-Multiplexer ve yazmaca yazma	$0.2 + 0.2 + 0.6 = 1$

> **Ders sonrası not:** EX için 1 ns, geleneksel dökümdeki 0.8 ns'den 0.2 ns fazladır. WB'de de D-Multiplexer ve yazmaya ek olarak bir 0.2 sayıldı. Bu fazlalık büyük olasılıkla araya eklenen pipeline yazmacının gecikmesidir. Altyazıda bu 0.2'nin kaynağı söylenmedi.

Aşama süreleri farklıdır. Fakat sistemin paralel çalışabilmesi için senkron olması, yani bütün aşamaların aynı anda başlayıp aynı anda bitmesi gerekir. Bu yüzden sistemin frekansı en yavaş aşamaya uyar. En uzun süre 1 ns olduğundan pipeline'da saat periyodu $T = 1$ ns olur.

7.3 Çalışma Çizelgesi

Yedi mikro işlem için çizelge şöyledir:

Mikro işlem	1	2	3	4	5	6	7	8	9
1	OF	EX	WB						
2		OF	EX	WB					
3			OF	EX	WB				
4				OF	EX	WB			
5					OF	EX	WB		
6						OF	EX	WB	
7							OF	EX	WB

- **1. çevrim:** 1. işlem için OF yapılır.
- **2. çevrim:** 1. işlem EX'e geçer. OF katmanı boşaldığı için 2. işlem OF'ye girer.
- **3. çevrim:** 1. işlem WB'ye, 2. işlem EX'e geçer, 3. işlem OF'ye girer. Bu çevrimin sonunda pipeline tamamen dolmuştur, bütün katmanlar iş yapar ve 1. mikro işlem biter.
- **4. çevrim:** 2. işlem WB, 3. işlem EX, 4. işlem OF.

Yedi mikro işlem dokuz çevrimde biter. (Hoca bir an "9 instruction" dedi, hemen düzeltti: 9 saat periyodunda 7 komut tamamlanır.)

7.4 Karşılaştırma

- **Pipeline:** $9 \times 1 = 9$ ns.
- **Geleneksel:** $7 \times 2.4 = 16.8$ ns.
- **Oran:** $16.8/9 \approx 1.87$.

Hoca bunu "neredeyse iki kat" diye yorumladı. Datapath'te bu büyük bir kazançtır.

Soru: Uzun işlemlerde aşama sayısını artırarak iki katından çok daha fazla hızlanamaz mıyız?
Cevap: Evet. Hoca, işlemlerin yalnızca OF, EX ve WB'den oluşmadığını, kontrol bloğundaki işlemlerin de ekleneceğini söyleyerek bir sonraki yapıya geçti.

Ek bilgi (derste anlatılmadı): Hocanın formülüyle n komut, k aşama ve t periyot için pipeline süresi $(k-1+n)\,t$, tek çevrim süresi T ile geleneksel süre nT olur. Oran $\frac{nT}{(k-1+n)t}$ olup, n büyüdükçe T/t değerine yaklaşır. Yani bu örnekte üst sınır 2.4 kattır; yedi işlemde 1.87'de kalınmasının sebebi pipeline'ın dolma süresidir.

Hocanın vurgusu: Bu tip sorular sınavda sorulabilir. Örneğin 7 yerine 15 mikro işlem, 3 yerine 5 aşama verilip geleneksel yöntemle pipeline'ın karşılaştırılması istenebilir.

8. Kontrol Birimiyle Dört Aşamalı Pipeline

Pipeline'a kontrol devresi de katıldığında dört aşama elde edilir:

- **1. aşama, IF (instruction fetch):** Komut belleğinden komutun okunması. "Fetch" alıp getirmek demektir.
- **2. aşama, DOF (decode and operand fetch):** Komutun çözülmesi ve komutta kullanılacak işlenenlerin getirilmesi.
- **3. aşama, EX (execution):** İşlemin yapılması.
- **4. aşama, WB (write back):** Sonucun belleğe ya da yazmaca yazılması.

Her aşamanın arasına bir yazmaç konur. Her katmanda elde edilen sonuç yazmaca kaydedilir, sonraki aşama bu yazmaçtan okur. Böylece önceki katman yeni bir işe başlamaya hazır olur. Her katmanın süresi en yavaş birime göre 1 ns alınır.

Karşılaştırma için slayttaki örnek şudur: R1'den R7'ye yedi yazmaca sırasıyla 1'den 7'ye kadar sabitleri yükleyen yedi komut (load the constant 1 through 7 into the 7 registers R1 to R7).

Mikro işlem	1	2	3	4	5	6	7	8	9	10
1	IF	DOF	EX	WB						
2		IF	DOF	EX	WB					
3			IF	DOF	EX	WB				
4				IF	DOF	EX	WB			
5					IF	DOF	EX	WB		
6						IF	DOF	EX	WB	
7							IF	DOF	EX	WB

1. işlem 4. periyodun sonunda, 2. işlem 5. periyodun sonunda biter ve böyle devam eder. Hocanın verdiği kestirme formül:

$$\text{toplam çevrim} = (\text{aşama sayısı} - 1) + \text{mikro işlem sayısı}$$

Buna göre $(4-1)+7 = 10$ çevrim. Formül üç aşamalı örneği de verir: $(3-1)+7 = 9$ (ara adım eklendi). Hocanın sınav tipi örneğindeki 15 mikro işlem ve 5 aşama için ise $(5-1)+15 = 19$ çevrim çıkar (ara adım eklendi).

- **Pipeline:** $10 \times 1 = 10$ ns.
- **Geleneksel:** $7 \times 3.4 = 23.8$ ns.
- **Oran:** $23.8/10 = 2.38$, hocanın ifadesiyle yaklaşık 2.4 kat.

Ders sonrası not: Bu derste üç ayrı geleneksel periyot geçti, karıştırmayın. 9.8 ns, 4 ns'lik gecikmelerle hesaplanan tek çevrimli bilgisayarın süresidir. 2.4 ns, yalnız datapath örneğinin süresidir. 3.4 ns, kontrollü pipeline karşılaştırmasında kullanılan değerdir. Hoca da pipeline örneğine geçerken sürelerin değiştiğini söyledi.

9. Pipeline Tehlikeleri

Pipeline performansı çok artırır, ama zorlukları da vardır. Bunlara tehlike (hazard) denir: veri bağımlılığı (data dependency) ve kontrol bağımlılığı (control dependency).

Hocanın vurgusu: Bu başlıkta önemli olan, problemi anlamak ve bir çözüm üretilmesi gerektiğini düşünmektir. Hoca çözümlerin ayrıntısı üzerinde çok durmadı.

9.1 Pipeline'ı Doldurma ve Boşaltma

Sıfırdan başlandığında pipeline dolana kadar çıktı üretilmez. Doldurulduktan sonra her saat periyodunda bir sonuç alınır. Fakat bir atlama işleminde program akışı sırayla gitmeyeceği için pipeline'ı boşaltmak gerekir. Bu da performansı düşürür.

9.2 Veri Tehlikesi

Arka arkaya verilen üç komutta veri bağımlılığı görülür. Kısa nottaki slayt satırına göre komutlar `MOVA R1, R5`, `ADD R2, R1, R6` ve `ADD R3, R1, R2`'dir. Birinci komutun hedefi R1 iken, ikinci komut R1 henüz güncellenmeden R1'i okumak ister:

- 1. komut sonucu R1'e **4. çevrimde** (WB) yazar.
- 2. komut işlenenini **3. çevrimde** (DOF) okur.

Yani sonuç yazılmadan ikinci satır yazmacın içeriğini okumaya çalışır. Aynı durum R2 için de vardır: 2. komut R2'ye 5. çevrimde yazar, 3. komut ise onu 4. çevrimde okur (ara adım eklendi).

Yazılım tabanlı çözüm (program based solution). Araya işlem yapmayan bir komut (no operation, NOP) eklenerek gecikme sağlanır. Böylece yazma ile okuma aynı periyoda denk gelir ve veri doğru okunur. Aynı işlem R2 için de yapılır:

Komut	1	2	3	4	5	6	7	8
`MOVA R1, R5`	IF	DOF	EX	WB				
`NOP`		IF	DOF	EX	WB			
`ADD R2, R1, R6`			IF	DOF	EX	WB		
`NOP`				IF	DOF	EX	WB	
`ADD R3, R1, R2`					IF	DOF	EX	WB

Çizelgeyi hoca sözlü anlattı; satırlar bu anlatımdan kuruldu ve python ile doğrulandı (ara adım eklendi). R1, 4. çevrimde hem yazılır hem okunur. R2, 6. çevrimde hem yazılır hem okunur.

> **Ders sonrası not:** Hoca NOP'u eklemeyi bir yerde "işlemci içerisindeki optimizer", dallanma sorusunda ise "işletim sistemi arka tarafta çalışan bir optimizer" diye anlattı. Yazılım çözümünde NOP'ları büyük olasılıkla derleyici veya assembler program hazırlanırken ekler, işletim sistemi çalışma anında eklemez. Hoca sonraki derste de bunun "compiler düzeyinde" çözülebileceğini söyledi.

Soru: NOP'u bir kez koymak her zaman yeter mi, yoksa arka arkaya iki üç NOP gereken yerler olur mu? **Cevap:** Hoca, bu örnekte birinin yettiğini düşündüğünü, fakat mimariye bakmak gerektiğini söyledi. Aşama sayısı daha fazlaysa yetmeyebilir. Dört aşamalı bu yapıda birden fazlası gerekmez. Hesap da bunu doğrular: i. komut $i+3$. çevrimde yazar, j. komut $j+1$. çevrimde okur, dolayısıyla $j - i \geq 2$ yeterlidir, yani aralarında bir NOP olması yeter.

Soru: NOP koymak sistemi yavaşlatmıyor mu? **Cevap:** Elbette yavaşlatır, pipeline'ın verimi düşer. Ama hocaya göre bu bağımlılık görülür, fakat çok sık değildir. Asıl önemlisi, karşılaşıldığında tedbir alınmazsa işlemcinin doğru çalışmayacak olmasıdır. Tedbir almak zorunludur.

Donanım çözümü. Devreye, EX aşamasında yapılan işlem sonucunu doğrudan ilgili yere aktaran bir blok yerleştirilir. Hoca bu tür çözümlerin olduğunu söyleyip ayrıntıya girmedi.

> **Ek bilgi (derste anlatılmadı):** Kısa nota göre slaytta başka bir donanım yaklaşımı da gösterilmiştir: Tehlike algılanınca pipeline bekletilir (stall) ve "bubble" gönderilir. Sonucu doğrudan aktarma yaklaşımının adı büyük olasılıkla iletmedir (forwarding). Bekletme gibi NOP de süre kaybettirir; iletme ise bu kaybı büyük olasılıkla azaltır.

9.3 Dallanma Tehlikesi

Slaydın sonraki kısmında iki NOP gerekir ve durum farklıdır. Birinci komut bir dallanma komutu olsun. Pipeline her zaman komut akışında bir sonraki satıra geçileceğini varsayar. Hocanın anlatımına göre birinci komut WB'ye geldiğinde dallanma olduğu ve yeni bir adrese gidileceği anlaşılır. Fakat bu sırada önceki aşamalarda sonraki komutlar yürütülmeye başlanmıştır: 2. komut EX'te, 3. komut DOF'tadır.

Oysa birinci komut, örnekteki gibi 20 adresine sıçramıştır. Bu durumda pipeline'a alınan 2. ve 3. komutların silinmesi, yani pipeline'ın boşaltılması gerekir. İzlenen yol, dallanma komutundan sonra iki NOP eklemektir. Böylece yanlış işlem yapılması önlenir.

> **Ders sonrası not:** Hoca "writeback'e geldiğinde anlıyorsunuz" dedi. 1. komut WB'deyken (4. çevrim) 2. komut EX'te, 3. komut DOF'ta, IF'te ise yeni komut vardır. Karar WB'nin sonunda verilseydi bu üç komutun da silinmesi gerekirdi; iki NOP'un yetmesi, hedef adresin 4. çevrimde IF'e girdiğini gösterir. Yani PC'nin büyük olasılıkla EX aşamasının sonunda yüklendiği anlaşılır. Hesap: EX'te karar → 2 yanlış komut, WB'de karar → 3 yanlış komut.

Soru: Program dallanmayı son aşamada anlıyorsa diğer işlemler arada yürümüş olmuyor mu? NOP koymaya ne zaman karar veriliyor? **Cevap:** Dallanma komutu görüldüğü anda araya iki NOP eklenir. Yukarıdaki anlatım, bu fark edilmeseydi ne olacağını gösterir.

Soru: Birinci çevrimde, IF'te kod okunurken dallanma olduğu fark edilip, nereye dallanılacağı beklenmeden NOP'lar eklenmiş olmuyor mu? **Cevap:** Evet, temel mantık budur. Komutun bir dallanma olduğu IF'te okunan yüksek anlamlı 7 bitten anlaşılır.

10. Giriş/Çıkış Birimleri ve Çevre Birimleri

İkinci yarının konusu giriş/çıkış ve iletişimdir. Giriş/çıkış birimleri denince bilgisayara bağlı klavye, ekran, yazıcı, manyetik disk, tarayıcı, modem, hoparlör ve mikrofon gibi birimler anlaşılır. Analog çalışan birimler için işlemciyle haberleşmede analogdan dijitale, çıkış birimlerinde dijitalden analoga dönüşüm gerekir; fakat bu dersin konusu değildir. Bilgisayara doğrudan (online) bağlı bu birimlere çevre birimi (peripheral) denir. Örnekler: klavye, hard disk, LCD ekran.

Klavye. Klavye bir tarama matrisi (scan matrix) üzerinden çalışır. Yapıda matrisi süren bir çoklayıcı ve bir çözücü (decoder) vardır. Hangi tuşa basıldığı, satır ve sütunun kesişiminden bulunur. Klavyenin içindeki mikrodenetleyici, basılan tuşun satır ve sütununa göre bir tarama kodu (scan code) üretir. Bu kodun hangi harfe denk geldiği bir tablodan işlemci tarafında anlaşılır. Hocaya göre mikroişlemciler dersinde bununla ilgili örnekler yapılacak.

Hard disk ve LCD ekran da benzer örneklerdir. Hoca G/Ç işlemcisine (I/O processor) yeri geldiğinde değineceğini söyleyip geçti.

11. G/Ç Arayüzü ve Portlara Erişim

11.1 Neden Arayüz Gerekir?

Çevre birimlerinin işlemciyle haberleşmesi için bir G/Ç arayüzü (I/O interface) gerekir. Arayüz, bağlantıda gerekli dönüşümleri yapar. Çünkü çevre birimleri çoğunlukla elektromekanik cihazlardır. Farklı elektriksel seviyeler kullanabilirler: Örneğin 0-5 V yerine 0-12 V aralığında çalışabilirler. Veri formatları ve veri yapıları da farklı olabilir. Bütün bunları düzenleyen bir ara birime ihtiyaç vardır.

11.2 Yollar ve Bağlantılar

Hocanın "çok şey anlatıyor" dediği temel şekilde üç yol vardır:

- **Veri yolu (data bus):** Çift yönlüdür (bidirectional).
- **Adres yolu (address bus):** Tek yönlüdür, işlemciden dışarıya doğrudur. Adres bilgisini üreten işlemcidir.
- **Kontrol yolu (control bus):** İşaretlerin bir kısmı giriş, bir kısmı çıkış yönündedir.

Her çevre birimi yollara bir arayüz üzerinden bağlanır:

- **Giriş birimi (ör. klavye):** Arayüzün veri yolu bağlantısı yalnızca işlemciye veri gönderme yönündedir. Adres bilgisi ve kontrol işareti arayüze giriş olarak gelir. Basılan tuşun tarama kodu veri yolu üzerinden işlemciye gönderilir.
- **Çıkış birimi (ör. CRT ekran, yazıcı):** Veri, veri yolundan dışarıya doğru aktarılır. Örneğin ekranda gösterilecek karakter böyle gönderilir.
- **Hem giriş hem çıkış yapan birim (ör. manyetik disk):** Veri yoluna çift yönlü bağlanır.

11.3 Portlara Erişim: Isolated I/O ve Memory-Mapped I/O

G/Ç cihazı denince portlar akla gelir. Portlara erişim için daha önce görülen iki teknik vardır:

Özellik	Isolated I/O	Memory-mapped I/O
Yollar	Adres ve veri yolu ortak, kontrol hatları farklı	Bellek ve G/Ç için ortak veri, adres ve kontrol yolu
Komut	Özel IN ve OUT komutları vardır	Ekstra komut kullanılmaz
Port görünümü	Ayrı G/Ç alanı	Portlar bellek adresi gibi kullanılır
Örnek	Intel 8086	Motorola işlemciler

12. Örnek Arayüzün İç Yapısı ve Adres Çözme

> **Hocanın vurgusu:** Bir G/Ç cihazının işlemciye bağlanmasında en önemli konu burasıdır. Çok dikkatli dinleyin.

12.1 İç Yazmaçlar ve Seçim Tablosu

Örnek arayüzde dört iç yazmaç vardır: Port A, Port B, kontrol yazmacı (control register) ve durum yazmacı (status register). Sayıları arayüze göre değişir; başka arayüzlerde 2, 10 ya da 15 olabilir. Soldaki uçların hepsi CPU'ya, sağdaki uçların hepsi G/Ç cihazına bağlanır.

> **Ders sonrası not:** Hoca bir yerde "iki tane olan bir yapı üzerinden konuşuyoruz" dedi. Örnekte dört iç yazmaç vardır; hocanın kendi saydığı liste ve aşağıdaki tablo da bunu gösterir.

Arayüzün içindeki zamanlama ve kontrol (timing and control) bloğu, gelen adres bilgisine göre arayüzün seçilip seçilmeyeceğine karar verir. Bunun için çip seçme (chip select, CS) ve yazmaç seçme (register select, RS1 ve RS0) uçlarına bakar:

CS	RS1	RS0	Seçilen
0	X	X	Hiçbiri; veri yolu yüksek empedansta (high-Z)
1	0	0	Port A
1	0	1	Port B
1	1	0	Kontrol yazmacı
1	1	1	Durum yazmacı

CS = 0 iken RS uçları ne olursa olsun arayüz seçilmez. Veri yolu üç durumlu (three state) olduğu için arayüzün veri yolu tarafındaki çıkışları yüksek empedansta kalır. CS = 1 olduğunda, RS uçlarının seçtiği yazmaca işlemci erişebilir.

12.2 Adres Çözme Örneği

Arayüz G/Ç uzayında (I/O space) bir adres aralığına yerleştirilir ve bu aralık istenen yere konabilir. Hocanın örneği A8h adresinden başlar. Önce "A8, A9" dedi, ama dört yazmaç olduğu için arayüz dört adres, yani dört bayt yer tutar. Bu yüzden aralık A8h, A9h, AAh, ABh'dir.

Adresler ikili yazılır:

Adres	A_7	A_6	A_5	A_4	A_3	A_2	A_1	A_0	Yazmaç
A8h	1	0	1	0	1	0	0	0	Port A
A9h	1	0	1	0	1	0	0	1	Port B
AAh	1	0	1	0	1	0	1	0	Kontrol

Adres	A_7	A_6	A_5	A_4	A_3	A_2	A_1	A_0	Yazmaç
ABh	1	0	1	0	1	0	1	1	Durum

Arka arkaya gelen dört adreste yalnızca düşük anlamlı iki bit, A_1 ve A_0 değişir. Bunlar iç yazmaçları seçmek için kullanılır, yani RS1 ve RS0'a bağlanır (CS'nin üretiminde bu bitler fark etmez). Değişmeyen A_7-A_2 uçları `101010` olduğunda arayüz seçilmelidir.

CS işareti bu sabit uçlardan üretilir. Bütün girişler 1 olduğunda çıkışı 1 olan kapı AND'dir. 1 olması gereken uçlar doğrudan, 0 olması gerekenler bir NOT kapısından geçirilerek bağlanır:

$$CS = A_7 \cdot \overline{A_6} \cdot A_5 \cdot \overline{A_4} \cdot A_3 \cdot \overline{A_2}$$

Böylece CS yalnızca adres yolunda A8h, A9h, AAh veya ABh göründüğünde 1 olur. 0-255 arasındaki bütün adresler için python ile denendi: CS yalnızca bu dört adreste 1'dir.

12.3 Yazmaçlara Erişim

Bu tasarımda 8086'nın porttan okuma komutu `IN AL, A8h` çalıştırıldığında, A8h adresindeki Port A yazmacı işlemcinin AL iç yazmacına okunur. Yazmaçlar belli adreslere yerleştirildiği için işlemci hepsine erişebilir, onları okuyabilir ve onlara istenen sayıyı yükleyebilir.

Portlar cihazla bağlantılı uçlardır. Örneğin Port B, klavyeden alınan tarama kodunu tutan yazmaç olabilir. O zaman `IN AL, A9h` komutu, klavyede basılan tuşun kodunu AL'ye okur.

13. Arayüz ile Cihaz Arasında Strobing ve Handshaking

Burada anlatılan haberleşme CPU ile arayüz arasında değil, **arayüz ile cihaz arasındadır**. Veri yolu, üç durumlu tamponlarla (three state buffer) iki birim arasında çift yönlü hale getirilmiş kabul edilir. Veri gönderen birime kaynak birim (source unit), alan birime hedef birim (destination unit) denir.

13.1 Strobing

Strobing'de veri yoluna ek olarak tek bir kontrol teli vardır. İki türlü olabilir:

- **Hedef tarafından başlatılan aktarım (destination initiated transfer):** Zamanlamaya bakıldığında önce hedef birim strobe işareti üretir. Bu işaret geldikten sonra kaynak birim veriyi gönderir. Verinin aktif olduğu aralıkta veri okunur.
- **Kaynak tarafından başlatılan aktarım (source initiated transfer):** Klavye örneği buna uyar. Kaynak birim klavye olsun, hedef birim arayüz. Tuşa basılınca veri yolunda veri oluşur ve kaynak aynı zamanda strobe gönderir. Strobe'u üreten de veriyi koyan da kaynaktır. Bu işaret "verim hazır" demektir. İşaret oluşunca hedef birim veri yolunu okur.

Sorun: İki türde de karşı tarafın veriyi okuduğuna dair hiçbir işaret gelmez. Tek taraflı bir bilgi aktarımı olur. Bu yüzden tek telli strobe işareti yeterli değildir.

> **Ders sonrası not:** Hoca konuya girerken "iki telli haberleşme, buna strobing işareti deniyor" dedi. Doğrusu tek teldir; hoca da anlatımın devamında "tek bir tane tel" ve "tek telli strob" dedi. İki telli olan handshaking'dir.

13.2 Handshaking

Handshaking'de tel sayısı ikiye çıkar: istek (request) ve yanıt (reply). Adı "el sıkışma"dan gelir. Derste hedef tarafından başlatılan örnek görüldü:

- Veriyi okuyacak olan hedef birim önce request gönderir.
- Kaynak birim buna karşılık veriyi veri yoluna koyar ve reply üretir. Reply, "hazır veri var" demektir.
- Hedef birim veri yolundaki veriyi okur, **okuduktan sonra** request'i kaldırır.

Son adım önemlidir: Request'in kalkması, hedefin veriyi aldığını kaynağa gösterir. Böylece talep gönderilmiş, karşı taraf hazırladığını bildirmiş, hedef de aldığını göstermiş olur. Bu yüzden iki telli handshaking daha sağlıklı bir iletişimdir.

14. Paralel ve Seri İletişim

İki iletişim biçimi vardır. Bu konular veri iletişimi (data communications) dersinde de anlatılacaktır.

Paralel iletişim: İşlemcideki paralel port üzerinden veri yolu genişliği kadar bit aktarılır. N bit için n tel gerekir ve n bitlik veri tek hamlede gider.

Seri iletişim: Bitler tek tel üzerinden arka arkaya gönderilir. Paralele göre daha yavaştır, ama tek tel kullanıldığı için daha ekonomik ve daha hafif bir çözümdür. Modem haberleşmesi seridir. USB portuna bağlı yazıcı ve diğer cihazlar da seri iletişimle haberleşir.

Seri iletişimin üç çalışma modu vardır:

Mod	Tanım	Hocanın örneği
Simplex	Veri yalnızca tek yönde taşınır	Radyo sinyalleri
Half duplex	İki yönde iletebilir, ama aynı anda yalnızca bir yönde; bir zaman diliminde çıkış, sonraki dilimde giriş yapılır	Telsiz
Full duplex	İki yönde aynı anda veri aktarılır	Hoca örnek vermedi

Dersin Toparlaması

Bu derste, önceki dersin datapath'ine komut kümesi ve kontrol birimi eklenerek tek çevrimli bilgisayar tamamlandı. 16 bitlik komutun 7 bitlik opcode'u hem işlemi hem adresleme modunu söyler. Komut çözücü, opcode'un dört bitinden kontrol işaretlerini üretir. Bir çevrimin süresi en yavaş yolun toplam gecikmesiyle belirlenir. Pipeline bu süreyi aşamalara böler: Aşama sonlarına yazmaç konur, periyodu en yavaş aşama belirler ve toplam çevrim (aşama sayısı $-$ 1) + işlem sayısı olur. Bunun bedeli veri ve dallanma tehlikeleridir; yazılımda NOP eklenerek, donanımda sonuç doğrudan aktarılarak çözülür.

İkinci yarıda işlemcinin dış dünyayla bağlantısına geçildi. Çevre birimleri bir arayüz üzerinden bağlanır, arayüzün iç yazmaçlarına CS ve RS uçlarıyla belirli adreslerden erişilir. Arayüz ile cihaz arasındaki haberleşme strobing veya daha güvenli olan handshaking ile yapılır. Seri iletişimde simplex, half duplex ve full duplex modları görüldü. Asenkron ve senkron seri haberleşme, start, stop ve parity bitleri sonraki derse kaldı. Ardından önbellek konusuna geçilecektir.

Not: İsterseniz bu kayıtları Gemini Notebook gibi bir araca atıp daha istifadeli hale getirebilirsiniz.